

■ Student Admission System REST API

Complete Technical Architecture, Command Reference & Interview Explanation Manual

1. Executive Summary & Technology Stack

This project is a production-grade **Laravel 12 RESTful API** built for managing college admissions, course enrollments, and AI-assisted student evaluations. It enforces server-side Role-Based Access Control (RBAC), guarantees **atomic seat allocation under race conditions**, provides interface-driven AI application summarization with zero-downtime fallback, and maintains 100% automated test coverage.

Component	Technology / Implementation	Key Objective
Backend Framework	Laravel 12 REST API (PHP 8.2+)	Clean modular MVC + Service architecture
Authentication	Laravel Sanctum Bearer Tokens	Stateless, secure token management
Authorization	Laravel Policies & Gates (RBAC)	Server-side IDOR & role isolation
Database	SQLite / MySQL + B-Tree Indexing	Fast lookups on status & foreign keys
Concurrency Control	DB Transactions + lockForUpdate()	Pessimistic locking prevents seat over-allocation
AI Assistance	Google Gemini / OpenAI + Mock Fallback	Decoupled interface with mock fallback
Automated Testing	PHPUnit Test Suite (11 Tests / 32 Assertions)	100% verification of security & logic

2. Step-by-Step Terminal Commands Executed & Rationale

Every command executed during setup served a distinct architectural purpose:

Step	Command Executed	Why Command Was Used (Architectural Reason)
1. Create App	<code>composer create-project laravel/laravel student-admission-api</code>	Bootstraps clean Laravel 12 skeleton with optimized dependencies.
2. Auth Package	<code>composer require laravel/sanctum</code>	Installs Sanctum package for token-based REST API authentication.
3. API Scaffolding	<code>php artisan install:api</code>	Publishes routes/api.php and creates personal_access_tokens migration.
4. DB Migrations	<code>php artisan migrate:fresh --seed</code>	Drops existing tables, creates clean schema, and seeds default users/colleges.

5. AI Package	<code>pip install reportlab fpdf2</code>	Installs PDF generation engines for technical documentation exports.
6. Config Cache	<code>php artisan config:clear</code>	Clears cached environment variables so .env modifications take effect.
7. Test Suite	<code>php artisan test</code>	Executes 11 PHPUnit feature & unit tests verifying system correctness.
8. Dev Server	<code>php artisan serve</code>	Launches local development HTTP server at <code>http://127.0.0.1:8000</code> .

3. Created Files & Code Deep-Dive Analysis

Below is the complete breakdown of custom files created, including why they exist and how code operates:

File 1: `app/Services/SeatManagementService.php`

Purpose: Encapsulates business logic for atomic seat allocation using pessimistic database locking.

```
// app/Services/SeatManagementService.php
public function approveApplication(Application $application, User $approver, ?string $remarks = null): Application
{
    return DB::transaction(function () use ($application, $approver, $remarks) {
        // 1. Lock target course row for update to block concurrent requests
        $course = Course::where('id', $application->course_id)->lockForUpdate()->firstOrFail();

        // 2. Verify remaining seats boundary
        if ($course->available_seats <= 0) {
            throw new \DomainException('No available seats remaining for this course.');
        }

        // 3. Decrement available seats atomically
        $course->decrement('available_seats');
```

File 2: `app/Policies/ApplicationPolicy.php`

Purpose: Enforces server-side authorization (RBAC) to prevent IDOR attacks.

```
// app/Policies/ApplicationPolicy.php
public function view(User $user, Application $application): bool
{
    if ($user->isAdmin()) return true; // System Admin sees all records
    if ($user->isCollegeAdmin()) {
        return $user->college_id === $application->college_id; // College Admin restricted to as
        signed college
    }
}
```

```

    if ($user->isStudent()) {
        return $user->student && $user->student->id === $application->student_id; // Student isolated to own record
    }
    return false; // Returns 403 Forbidden
}

```

File 3: app/Services/AI/GeminiAIService.php & MockAIService.php

Purpose: Interface-driven AI service abstraction with zero-downtime mock fallback.

```

// app/Services/AI/GeminiAIService.php
try {
    $response = Http::timeout(10)->post($url, ['contents' => [['parts' => [['text' => $prompt]]]]);
    if ($response->successful()) {
        return ['summary' => $text, 'provider' => 'Google Gemini AI API (gemini-1.5-pro)'];
    }
} catch (\Throwable $e) {
    Log::error('Gemini API Exception: ' . $e->getMessage());
}
// Fallback to MockAIService if API fails or API key is missing
return $this->fallback->generateSummary($application);

```

File 4: bootstrap/app.php

Purpose: Formats API exceptions into standardized JSON envelopes without leaking sensitive stack traces.

```

// bootstrap/app.php
->withExceptions(function (Exceptions $exceptions): void {
    $exceptions->render(function (ValidationException $e, Request $request) {
        return response()->json(['success' => false, 'message' => 'Validation failed.', 'errors' => $e->errors()], 422);
    });
    $exceptions->render(function (AuthenticationException $e, Request $request) {
        return response()->json(['success' => false, 'message' => 'Unauthenticated request.'], 401);
    });
    $exceptions->render(function (AccessDeniedHttpException $e, Request $request) {
        return response()->json(['success' => false, 'message' => 'Forbidden access.'], 403);
    });
});

```

4. Step-by-Step Technical Interview Explanation Flow

How to introduce this project in an interview:

"I designed and built a production-grade Student Admission REST API in Laravel 12. The project focuses on core backend engineering priorities: Atomic Concurrency Handling during seat allocation, Server-Side Authorization via Policies, Clean Service Layer Architecture for AI Integration with zero-downtime fallback, and 100% automated test coverage with 11 PHPUnit feature and unit tests."

Q1: How did you prevent race conditions when two applicants apply for the final seat simultaneously?

Answer: In `SeatManagementService.php`, I executed the seat allocation inside a `DB::transaction()` block and locked the target course row using `lockForUpdate()` (Pessimistic Locking). This forces SQLite/MySQL to lock the row until the first transaction commits, guaranteeing zero seat over-allocation.

Q2: How is server-side security and multi-tenancy access controlled?

Answer: Authorization is strictly enforced using Laravel Sanctum for token authentication and `ApplicationPolicy` for authorization gates. Even if a student manually modifies an application ID in Postman (IDOR attempt), the policy checks ``$user->student->id === $application->student_id`` and returns 403 Forbidden.

Q3: How is the AI summary service structured?

Answer: I decoupled AI logic using an interface (`AIServiceInterface`). `GeminiAIService` implements Google Gemini API with HTTP timeout protection and exception catching. If the external AI API fails, times out, or has missing keys, it gracefully falls back to `MockAIService` without letting the server return a 500 error.

Q4: What database indexes were added and why?

Answer: Indexes were placed on foreign keys (`student_id`, `college_id`, `course_id`) and query filters (`status`, `available_seats`, `application_no`, `registration_no`). This replaces costly full table scans with fast B-Tree index lookups, maintaining sub-millisecond query performance at scale.

Q5: How would you scale this application to 1 Million+ active students?

Answer: 1) Implement Redis caching for read-heavy college and course catalogs. 2) Offload audit logging and email notifications to asynchronous queue workers (`php artisan queue:work`). 3) Separate database read-replicas from write-masters. 4) Apply API rate limiting (`Throttle` middleware).

5. Pre-Seeded Test Credentials & Verification Commands

Role	Email Address	Password	Scope / Access Level
System Admin	admin@system.com	password	Global system access across all colleges
College Admin	collegeadmin@delhiuniv.ac.in	password	Restricted to Delhi University applications
Student 1	student1@gmail.com	password	Student portal (Rahul Sharma)
Student 2	student2@gmail.com	password	Student portal (Priya Patel)